SOFTWARE DEV

MERN STACK DEVELOPMENT REPORT

Week 3 day 5 : Mini project a Task Tracker

Muneeb Ur Rehman

MERN STACK INTERN

1. Executive Summary

This report analyzes the architectural design and technical implementation of the Day 5 **Task Tracker** application. Built using pure Vanilla JavaScript, the application completely bypasses heavy modern frameworks while achieving optimal performance, maintainability, and responsiveness. The engineering core relies on two primary pillars: **State-Driven Rendering** (maintaining an array-based single source of truth) and **Event Delegation** (minimizing memory overhead through unified event handling).

2. Core Architectural Concepts

2.1 State Management (The Array Data Layer)

Traditional DOM manipulation often treats the UI as the source of truth, reading text directly from elements to calculate states. This architecture explicitly decouples data from the presentation layer:

JavaScript
```
let tasks = [];
```

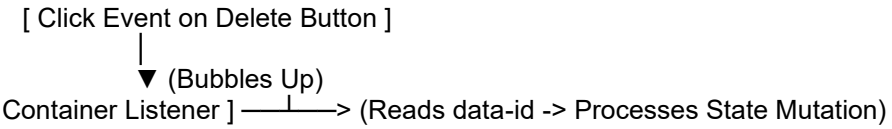
- **Single Source of Truth:** The `tasks` array contains structural objects representing application states:

  $$Task = \{id: String, text: String, completed: Boolean\}$$

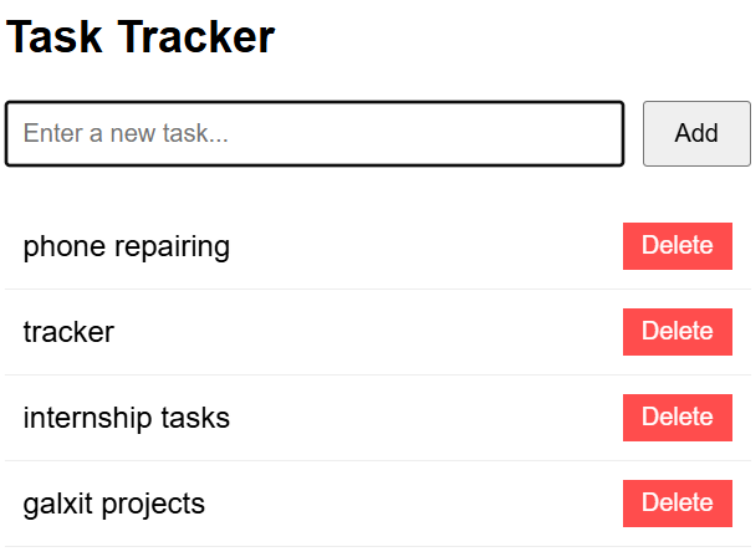
- **Immutability Strategy:** When mutating operations occur (e.g., updating a status or deleting a record), the application utilizes array methods like `.map()` and `.filter()` to update state structures cleanly before triggering UI updates.

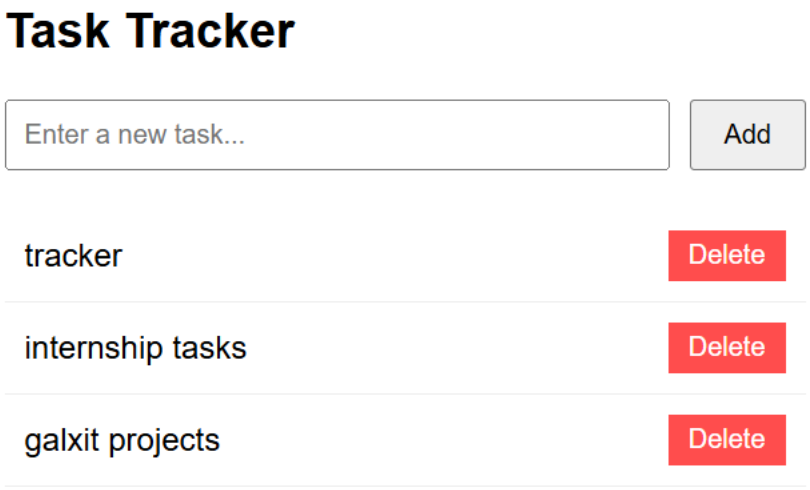
2.2 Event Delegation

Instead of appending event listeners to individual buttons or list elements—which can degrade performance and cause memory leaks during rapid creation/destruction cycles—the design anchors a single listener to the parent element container (`#taskList`).



As click events bubble up the DOM tree, the parent interceptor parses the event target to execute conditional mutations, maximizing CPU and memory performance.





3. Engineering Implementation Details

3.1 Structural Blueprint (index.html)

The interface leverages semantic HTML blocks paired with highly responsive layout systems.

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Day 5 Technical Implementation: Task Tracker</title>
    <style>
        :root {
            --primary: #2563eb;
            --danger: #ef4444;
```

```
            --text-main: #1f2937;
            --text-muted: #9ca3af;
        }
        body {
            font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
            max-width: 450px;
            margin: 50px auto;
            color: var(--text-main);
            padding: 0 15px;
        }
        .input-group {
            display: flex;
            gap: 10px;
            margin-bottom: 25px;
        }
        input {
            flex-grow: 1;
            padding: 10px 14px;
            border: 1px solid #d1d5db;
            border-radius: 6px;
            font-size: 1rem;
        }
        button#addTaskBtn {
            background: var(--primary);
            color: white;
            border: none;
            padding: 10px 18px;
            border-radius: 6px;
            cursor: pointer;
            font-weight: 500;
        }
        .task-item {
            display: flex;
            justify-content: space-between;
            align-items: center;
            padding: 12px;
            border-bottom: 1px solid #e5e7eb;
            animation: fadeIn 0.2s ease-in-out;
        }
        .task-text {
            cursor: pointer;
            flex-grow: 1;
            user-select: none;
        }
        .done {
            text-decoration: line-through;
            color: var(--text-muted);
        }
        .delete-btn {
            background: var(--danger);
            color: white;
            border: none;
            padding: 6px 12px;
            border-radius: 4px;
            cursor: pointer;
            font-size: 0.875rem;
        }
        @keyframes fadeIn {
            from { opacity: 0; transform: translateY(-5px); }
            to { opacity: 1; transform: translateY(0); }
        }
    </style>
</head>
<body>

    <h2>Task Management System</h2>

    <div class="input-group">
        <input type="text" id="taskInput" placeholder="Write a systemic task..." autocomplete="off">
```

```
        <button id="addTaskBtn">Add Task</button>
    </div>

    <div id="taskList"></div>

    <script src="app.js"></script>
</body>
</html>
```

3.2 Logic & Rendering Layer (app.js)

The application logic focuses entirely on standard array processing and element manipulation.

```
let tasks = [];

// DOM Element References
const taskInput = document.getElementById('taskInput');
const addTaskBtn = document.getElementById('addTaskBtn');
const taskListContainer = document.getElementById('taskList');

/**
 * Renders UI directly from the current state array.
 * Clears old elements to ensure the DOM matches the array state exactly.
 */
function renderTasks() {
    // Purge old view records to reset DOM state
    taskListContainer.innerHTML = '';

    if (tasks.length === 0) {
        taskListContainer.innerHTML = `<p style="color: #9ca3af; text-align: center;">No current tasks pending.</p>`;
        return;
    }

    // Programmatically construct DOM nodes from state records
    tasks.forEach((task) => {
        const taskDiv = document.createElement('div');
        taskDiv.className = 'task-item';

        // Inject structural templates mapped to item identifiers via data attributes
        taskDiv.innerHTML = `
            <span class="task-text ${task.completed ? 'done' : ''}" data-id="${task.id}">
                ${escapeHTML(task.text)}
            </span>
            <button class="delete-btn" data-id="${task.id}">Delete</button>
        `;

        taskListContainer.appendChild(taskDiv);
    });
}

/**
 * Handles creation mutations and adds new objects to the state array.
 */
function handleAddTask() {
    const text = taskInput.value.trim();
    if (text === '') return;

    const newTask = {
        id: Date.now().toString(), // Unix timestamp unique identifier
        text: text,
        completed: false
    };

    tasks.push(newTask);
    taskInput.value = ''; // Reset UI field input
    renderTasks();
}
```

```
/**
 * Centralized Parent Event Listener (Event Delegation Mechanism)
 */
taskListContainer.addEventListener('click', (e) => {
    const target = e.target;
    const id = target.getAttribute('data-id');

    // Exit early if the click occurred outside tracked target elements
    if (!id) return;

    if (target.classList.contains('task-text')) {
        // Condition A: Toggle Complete State via Immutable Array Map
        tasks = tasks.map(task =>
            task.id === id ? { ...task, completed: !task.completed } : task
        );
    } else if (target.classList.contains('delete-btn')) {
        // Condition B: Filter Item out of State Array
        tasks = tasks.filter(task => task.id !== id);
    }

    // Refresh layout view to map the updated state data array
    renderTasks();
});

/**
 * Primary Interaction Triggers
 */
addTaskBtn.addEventListener('click', handleAddTask);
taskInput.addEventListener('keypress', (e) => {
    if (e.key === 'Enter') handleAddTask();
});

/**
 * Utility function to secure inputs against XSS vulnerabilities
 */
function escapeHTML(str) {
    return str.replace(/[&<>'"]/g,
        tag => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', "'": '&#39;', '"': '&quot;' }[tag] || tag)
    );
}
```